

SEED

Loaves & Fishes — from a few seeds, multitudes

Custom wake words for Home Assistant from a handful of recordings of your own voice, multiplied into a rich dataset by voice-timbre transformation.

Free method for the community · **WORLD vocoder** engine · Trained locally on an **NVIDIA GPU**
Result: **frr 0.0155** (detects ~98.5%) with **0.187** false positives/hour — from just **180 samples**

SEED — Custom wake words for Home Assistant from a few recordings

(Loaves & Fishes — from a few seeds, multitudes)

Prologue — or why you end up training wake words

I have a job where I've closed the circle: the people above me and the people below me don't miss me. So I spend my time looking busy with things nobody else understands, so they don't realize it's by me, for me, and with me.

This is one of those things.

It started because I wanted my Home Assistant to answer to a name I chose, in my own voice, running on hardware that's mine — no cloud, no subscription, nobody's servers but my own. What follows is the method I arrived at, the machine I built it on, and every wall I hit along the way (there were many, and more than once I wanted to throw in the towel).

Maybe you have another reason. Maybe you want to resurrect the voice of a caring secretary who greets you when you arrive. Maybe a voice that gives some meaning to an *Office Space* kind of job, or that puts a face on your own *Zero Theorem* — that project where you chase a meaning that maybe only exists because you give it one. Doesn't matter which — the method is the same, and you don't need to be a career programmer. If I could put it together in odd spare moments, so can you.

One warning, though: along the way you'll find yourself with nowhere to hold on more than once. Like Mad Max in the middle of the desert without a drop of fuel to keep the journey going, there are stretches where the tools break, the disk fills up, the machine won't boot and there's nowhere to pull more from. There'll be *Downfall* moments, seeing no way out. You trip, fall and get back up more times than in *Heartbreak Ridge* — many hours of that. You get out by scraping from wherever you can (I resurrected a dead computer with two second-hand cards).

But in the end, when you're about to throw it all away, that thread of light appears — the one Hellboy kept hidden behind his cigar — the solution that was in front of you the whole time, you just couldn't see it. The *pitfalls* section below is the map of every place I got stuck, and how I moved the cigar aside each time, so you arrive with a full tank.

Where this comes from (project genealogy)

SEED wasn't born from nothing: it's a **spin-off from a larger main project**. That main project in turn spun off **HA.LAB**, set up initially for a **multi-room audio** (whole-house/venue sound) purpose. From there, the audio work led to the **Home Assistant Voice controller** — and once into voice, the need for a custom wake word appeared. Which is where this method comes in.

I mention it because these things almost never come out of a clean plan: you set out to pipe music around the rooms and you end up training neural networks so a gadget answers to your voice. Each step opened the next one.

Summary

Instead of recording hundreds of samples of your wake word, you record **~5-9 short "acted" takes** (different emotional registers of your own voice), and a script multiplies each into ~20 timbres (child / woman / man / old-person × speeds) using the **WORLD vocoder** (independent pitch and formant manipulation). The result is ~100-180 clean, varied samples from a few minutes of recording. The microWakeWord trainer then augments those to 50,000 and trains the model.

Core idea: a few well-chosen **seeds** + automatic expansion beats brute-force recording. Less effort, same result (or better).

Measured result: with **180 samples** SEED (9 recorded registers × 20 timbres) the final quantized model gave **frr = 0.0155 (detects ~98.5%) with 0.187 false positives per hour** (cutoff 0.98) — equal to or better than an earlier model of mine that needed **314 hand recordings**. Less recording, same result (or better).

Why I did it

I trained a custom wake word for my Home Assistant Voice PE. The first attempt used **314 real recordings** of my voice. It worked great, but recording 314 times is torture, and I suspected most were redundant.

The microWakeWord base method is 100% synthetic (Piper TTS). Real voice samples are an **add-on** that reduces false triggers and anchors the model to how YOU say the word. The community consensus is that ~30 real samples is usually enough. So 314 was overkill.

The question became: **can I get the variety of hundreds of recordings from a few, by transforming my voice into different timbres?** That's Loaves & Fishes (from few, multitudes).

The concept: two axes

The trick is separating two kinds of variation:

Axis B — REGISTERS (you record these, acting)

Different emotional/prosodic states of your own voice. A filter CANNOT invent these — the melody, pauses and emphasis come from your mouth. Choose registers **spread apart in the energy×tone spectrum** to cover the map without overlapping:

Register	Energy	Tone/speed	Spectrum zone
Tired	low	low-slow	bottom corner
Neutral	medium	normal	center
Rushed	high	fast	fast lateral
Lively	high	bright-high	top corner
Worked-up	very high	strong-high	top extreme

You record ~5 (2-3 takes each for natural variation → ~10 recordings total).

Axis A — TIMBRES (a script generates these)

Each register is multiplied into different "throats" — child, woman, man, old-person — plus speed variants. This is pure signal processing on your recording.

Timbre	What it does	Variants
Your voice	3 tones × 3 speeds	9
Child	high formant, moderate pitch	3
Woman	mid-high formant/pitch	3
Man	lower formant/pitch	3
Old	low pitch + tremor	2
Total		20

The multiplication

```

~10 recordings (registers) × 20 timbres = ~180 clean samples
                                   | (trainer augmentation)
                                   50,000 final samples

```

Two expansions from a few minutes of recording. That's the whole idea. Hence the name: from a few seeds, multitudes.

The two tables in detail

These are the actual tables of the method. **How to read them:** each 1 is a generated variant; ✕ marks a cross that's skipped (avoids odd voices and false triggers). Tone and speed already live INSIDE each register (a "tired" take is already low and slow), so they aren't crossed again.

Table 1 — TIMBRES (script-generated; each of your registers is multiplied by these 20 throats):

Timbre	Fast	Norm.	Slow	High	Norm.	Low	Total
Normal voice	1	1	1	3	3	3	9
Child	1	1	1	x			3
Woman	1	1	1	x			3
Man	1	1	1			x	3
Old	1		1			x	2
							20

Table 2 — REGISTERS (you record these, acting; moods spread across the energy×tone map without overlapping):

Register	Fast	Norm.	Slow	High	Norm.	Low	Total
Tired/apathetic			1			1	1
Relaxed/casual		1			1		1
Rushed	1			1			1
Lively/bright		1		1			1
Worked-up	1			1			1
							5

The payoff: 5 registers × 20 timbres = **100 samples** → with 2 takes each → **180** → augmentation → **50,000**. From a handful of recordings, a huge dataset.

Why the WORLD vocoder (and NOT a simple pitch-shift)

My first version used `pyrubberband` (pitch + tempo). Problem: shifting the pitch up to make a "child" turned my voice into a **chipmunk** — unnatural, and the model would learn a cartoon voice.

The fix was switching to the **WORLD vocoder** (`pyworld`), which decomposes the voice into three independent components: - **F0** — the pitch - **SP** — spectral envelope = the FORMANTS (the real timbre / throat size) - **AP** — aperiodicity (the "breath" component)

By moving the **formants separately from the pitch**, a child sounds like a child (small throat) instead of you sped up. It's the same principle as the formant filters in pro "voice changers."

Tuning key: for a believable child, raise the **formant warp**, not the pitch. High pitch alone = chipmunk. High formant = child's body. That insight fixed everything.

What the augmentation does (and what you must NOT duplicate)

The microWakeWord trainer augments your clean samples up to 50,000 by applying room reverbs (RIRs), background noise and volume scaling. This means:

- **Distance and volume are handled automatically** — do NOT add them in your script. If you pre-apply a "far" reverb and then the augments adds another, you get "far of far" — an unreal echo that hurts the model.
- **The 50,000 is a FIXED target.** It doesn't grow with your samples. The formula is:
$$\text{environments per sample} = 50,000 / (\text{your clean samples})$$
 Fewer clean samples → each is recreated in more environments. More samples → more content variety, fewer environments each. You control the balance between *content variety* (your job) and *environment variety* (the machine's job).

Rule: your script handles what changes the VOICE (timbre, character). The augmentation handles what changes the ENVIRONMENT (room, distance, noise, volume). Don't cross them.

Also: don't cross the tone/speed axes onto the registers. A register ALREADY is a tone+speed combination (a "tired" take is already low+slow). Forcing a "tired-fast-high" sounds fake.

The build: what machine this runs on (or how I resurrected an Area 51 Alienware)

All of this runs on a machine I rescued: an **Alienware R2** that was dead as a doornail. No matter how I tried to revive it by normal means (reverse engineering included), no luck... until I plugged in **two NVIDIA Titan RTX 24 GB** and pushed it with a **40-thread CPU**. The beast came back to life. It was like topping off an Egyptian pyramid with an elevator and air conditioning.

The machine, which I call **TITAN**, ended up like this:

- **Linux Mint** (Cinnamon).
- **2× NVIDIA Titan RTX** (24 GB each, on NVLink) — bought second-hand when nobody wanted them. Total overkill for this; a single decent GPU is enough.
- **40-thread CPU** pushing the sample generation (which runs on CPU, not GPU — it matters more than you'd think).
- Remote access via **VNC + noVNC** (to tinker from another machine's browser).
- **Docker** + nvidia-container-toolkit (the trainer runs in a container).
- A separate **1 TB NVMe** mounted by UUID in `/etc/fstab`, just for the trainer's data (the negative datasets take ~190 GB; see pitfalls).

That said — none of this is required. Real minimum: **an NVIDIA GPU with CUDA and ~6 GB VRAM** (a GTX 1060 works). My resurrected tower is overkill; I built it that way because I could and it was fun. Training needs a GPU; *running* the finished model only needs the Voice PE's ESP32 (a cents-worth chip).

To move files between my recording PC (Windows, with Audacity) and TITAN, I set up a **Samba shared folder**: I record the registers on Windows, drop them in the share, and the script processes them on TITAN.

How the training system is set up locally (in broad strokes)

All the training runs on your own machine, no cloud. Broadly, the setup is this:

1. **OS and drivers.** Linux (I use Mint). You install the proprietary NVIDIA drivers and the **CUDA toolkit**, so the GPU is available for compute. You check that `nvidia-smi` sees the card.
2. **Docker + GPU support.** You install **Docker** and the **nvidia-container-toolkit**, which is what lets a container use the host's GPU. Without it, the container can't "see" the card.
3. **The trainer in a container.** All the training software (TensorFlow, the Piper sample generator, the augments, the scripts) comes packaged in a **Docker image** — the microWakeWord Trainer one. You don't install that mess by hand; you launch the container and it's all inside. *Pin a working image version* (v11 works for me; v13/v14 crashed — see pitfalls).
4. **A big disk for the data.** The trainer downloads and processes a huge "negative" dataset (thousands of audio clips to teach the model what your word is NOT). That takes up a lot (~190 GB for me). Mount a separate disk and point the container's data volume there, to avoid filling the system disk. Mount it by **UUID** in `/etc/fstab` (the `/dev/sdX` or `/dev/nvmeX` name changes between reboots; the UUID doesn't).
5. **The generator voices.** The trainer creates tens of thousands of synthetic samples of your word with **Piper TTS** voices. For Spanish, you download the `.onnx` voices (from `rhasspy/piper-voices`) and drop them in the generator's voices folder. You can add several for more speaker diversity.
6. **Your personal samples.** The samples your script generates (the registers × timbres) get copied into the `personal_samples/` folder inside the container. They're the "real touch" added to the synthetic ones.
7. **Interface.** The trainer has a **web UI** (a local port) where you pick word, language and start. It can also be launched from the command line (sometimes more reliable — the web stalled on me). Progress is followed through the container's **log**.
8. **The process, once launched**, goes on its own through phases: **generates** the 50,000 synthetic samples → **augments** them (adds reverbs, noise, distances) → **trains** the model (this is where the GPU works hard) → **calibrates** and outputs the final `.tflite`.
9. **Remote access (optional).** Since the training machine usually sits in a corner, it's handy to have **VNC** (to see the desktop) or just **SSH** (to launch commands). I use VNC + noVNC to tinker from another machine's browser.

The result of all this is a `.tflite` file a couple hundred KB in size. That file is the only thing that travels to the Voice PE — where it runs on an ESP32 microcontroller with none of the above needed.

Step by step (what I actually did)

1. Record your registers

In Audacity, on a quiet machine, record ~5 registers (neutral, tired, rushed, lively, worked-up), 2-3 takes each. Export each as WAV. Record **in silence** — WORLD is sensitive to background noise. Name them `neutro.wav`, `cansado.wav`, etc.

2. Run the timbre expansion script

Install deps:

```
pip install soundfile pyworld numpy --break-system-packages
```

Run:

```
python3 panes_y_peces_world.py ./input ./output
```

It reads each register and writes ~20 timbre variants per register (child, woman, man, old × speeds), normalized to 16kHz/16-bit mono. From 9 recordings I got 180 clean samples.

3. Load them into the trainer

Copy the samples into the trainer's `personal_samples/` folder:

```
sudo docker cp ./output/. wakeword_trainer:/data/personal_samples/
```

4. Train

Set your wake phrase, choose language (Spanish worked once I installed the Spanish Piper voices — see pitfalls), and start. The pipeline: generate 50k TTS samples → augment → train → calibrate → produce a `.tflite`.

5. Deploy to the Voice PE

Build a correct microWakeWord manifest JSON (the trainer's `detection_calibration.json` is NOT the HA manifest), put the `.tflite` + `.json` in `config/www/wake_words/`, reference it in the ESPHome YAML, compile, flash. It coexists with the factory wake words.

The `.tflite`: how it comes out and how to get it to fit

This is one of the most confusing parts, so I'll detail it.

How it COMES OUT of the trainer. When done, the trainer leaves two files in its results folder: - The model: a `.tflite` (named something like `stream_state_internal_quant.tflite`), a couple hundred KB. - A `detection_calibration.json` with the calibration metrics (recall/faph per cutoff).

Big trap: that `detection_calibration.json` is **NOT** the manifest Home Assistant / ESPHome expects. If you try to use it as-is, it won't work. They're different things: one is metrics, the other is the model's config sheet.

What you have to prepare (BEFORE). You pull the `.tflite` out of the container and **hand-build** a separate manifest JSON, with the fields microWakeWord expects. Essentially it has: - `type`: "micro" - your word's name/ `wake_word` - `model`: the path to your `.tflite` — **with the `./` prefix** (this is key, see pitfalls: without the `./` ESPHome rejects it) - `version`: 2 - a `micro` block with: `probability_cutoff` (I use 0.98-0.99, higher = more selective), `sliding_window_size` (e.g. 4), `feature_step_size` (10), `tensor_arena_size` (memory reservation; see below) and the minimum ESPHome version.

Rename your `.tflite` to something of your own (e.g. `my_wakeword.tflite`) and have the manifest's `model` point to that exact name. The `.json` and the `.tflite` go **together in the same folder**.

The `tensor_arena_size`. It's the RAM the model reserves on the ESP32. You can set a generous value (I had plenty to spare: I reserved ~45 KB and the real model used ~16 KB). If compiling gives "Could not allocate tensor arena", raise it.

What happens AFTER (on flashing). A couple of things will happen: - After flashing, the device's wake-word selector goes **empty** ("No wake word") — you have to reselect yours by hand. - Your word **coexists** with the factory ones (Okay Nabu, Hey Jarvis...) — they all show in the dropdown. If you want only yours, it's a matter of select/remove in the config.

Compiling the firmware: pros, cons and the shortest path

For your wake word to get into the device, you have to **recompile the Voice PE firmware** and install it. There are several routes, and picking the right one saves you hours:

Route A — Compile on Home Assistant itself (ESPHome add-on). - *Pro*: most integrated; if your HA has plenty of RAM, it compiles and flashes over OTA without leaving the interface. - *Con*: if your HA is modest (a Yellow with CM4, a Raspberry...), the compiler runs **out of memory** and dies (`cc1plus: Killed`). You can force it (stopping add-ons, one process at a time), but it's suffering.

Route B — Compile in an ESPHome Docker on another machine with more RAM. - *Pro*: no memory choking; the powerful machine does the heavy lifting. You reuse the same machine you

train on. - *Con:* you have to set up the container and **pin the ESPHome version** to match your HA's (otherwise it complains of config incompatibility). A bit more initial prep.

Route C — Factory firmware + upload only the model. - Doesn't always apply, but if you only change the model and don't touch the rest of the config, sometimes it's enough to serve the `.tflite` + `.json` and reference them, without recompiling everything from scratch.

The shortest path (what I recommend): 1. If your HA has ample RAM → **Route A**, and go. 2. If your HA is modest → **don't fight its RAM**: set up **Route B** (ESPHome Docker on the training machine) from the start. What you lose setting it up you gain by not slamming into `cc1plus: Killed` over and over. 3. Keep the **factory firmware** (`.factory.bin`) ready to restore over USB if a flash goes wrong — that's your safety net.

(Setting up ESP32 compilation in Docker cleanly could well be a mini-project of its own — see the genealogy; these things always spin off into the next one.)

The pitfalls (the honest part — this is where the days went)

These are the bumps that cost me real time, and more than once made me think of quitting. If you hit them, you're not doing it wrong — the tools have sharp edges.

Spanish TTS voice bug

With `language=Spanish`, the trainer crashes downloading the Spanish voice (`Request.__init__() got unexpected keyword 'headers'` — a broken downloader). **Fix:** manually download Spanish `.onnx` voices from HF `rhasspy/piper-voices` and put them in `piper-sample-generator/voices/`. The trainer finds them and never touches the broken downloader. (You can add several — `es_ES`, `es_AR`, `es_MX` — and it uses all matching `es_*.onnx`, more speaker diversity.)

The disk fills up (twice)

The negative-dataset prep (FMA→WAV resample of 8000 files) filled my root SSD **twice** with cleanup flags off (it keeps ZIP + extracted + WAV). Point the trainer's data volume at a big disk. Mount it by **UUID** in fstab — the `/dev/nvmeX` name shuffles between reboots.

Trainer image versions

Image `v11` works. `v13` / `v14` crash on startup with a `dirname / NoneType` bug. Pin v11.

The generator hangs at 50000/50000 (with ONNX voices)

The progress-wrapper process spins forever (busy-wait on a dead child = zombie) after generating the last sample, never advancing to augmentation. **Rescue:** kill the wrapper, write the expected tag into `work/last_wake_word` (`wakeword:50000:es+voice1.onnx+voice2.onnx+...`), and relaunch — the trainer sees the samples already exist ("Sample generation not required") and jumps

straight to augmentation. Saves ~80 min of regeneration. (It happened to me twice; the rescue works both times.)

RAM to compile the firmware (the HA Yellow CM4)

I compiled the Voice PE firmware on a **Home Assistant Yellow with Compute Module 4 (CM4)** — a RAM-modest machine. The compile died with `cc1plus: Killed` (out of memory), always in the `esp-tflite-micro` files. In the end it **DID compile fine on the Yellow itself**, but by **killing processes and freeing temporary resources**: stopping all add-ons to free RAM and limiting the compile to one process at a time. Ninja resumes where it left off — don't "clean build files", or you start from zero.

That said, doing it that way is suffering. **The ideal is not to depend on the Yellow's RAM**: set up the compile in an **ESPHome Docker on another machine with more memory** (e.g. the same one you train on). Pin the ESPHome image version to match your HA's, give it the device YAML + the wake-word files, and compile there without choking. That could be a separate step of the project — setting up the ESP32 firmware compiler in a container, separate from the trainer.

The ESPHome manifest path

The `"model"` field in the manifest JSON needs the `./` prefix (`"model": "./mymodel.tflite"`) or ESPHome rejects it as "not a valid model name, local path, http(s) url". HTTP URLs also fail in current ESPHome — use a relative path with the `.json` and `.tflite` together.

The wake-word selector resets after flashing

After flashing, the Voice PE's selector goes to "No wake word" (empty) instead of your custom word — set it by hand or the device hears nothing.

WORLD hates fast/noisy recordings

My "rushed" take generated incompletely — WORLD's F0 detection struggles with very fast or noisy speech. Record a second take, or record the rushed one a bit cleaner.

Pros and cons (my honest take)

Pros

- **Way less recording.** 9 takes vs 314. Minutes instead of an afternoon of misery.
- **Lots of variety with little effort.** Each register becomes 20 timbres; the augments make 50k. Double expansion from a seed.
- **WORLD gives natural voices** — child/woman/old believable, not chipmunk.
- **You keep YOUR pronunciation.** Unlike pure TTS (which changes the whole voice), the timbres keep your prosody and accent — the model still learns how you say it.
- **Reproducible and editable.** The timbre table is just numbers; you add a voice by adding a row.

Cons / caveats

- **Needs an NVIDIA GPU** to train (running the model needs nothing — it's on the ESP32).
 - **WORLD is slower** than simple pitch-shift and sensitive to recording noise.
 - **The trainer has sharp edges** (see pitfalls). Expect to fight the tooling.
 - **Extreme timbres backfire**. Demon/ogre/robot voices increase false positives — the model learns to trigger on voices that aren't yours. Keep timbres in the human range.
-

Files

(Fill in your links) - `panes_y_peces_world.py` — the timbre expansion script - Example register recordings - The trained `.tflite` + manifest (as a reference)

Credits

- Built on TaterTotterson's microWakeWord Trainer
- WORLD vocoder via pyworld
- Piper voices from rhasspy/piper-voices

Method and script shared freely for the community. Use, adapt, improve.